El Sewedy University of Technology
Faculty of Computer Science & Engineering
CET251: Artificial Intelligence

PROJECT DESIGN REPORT

Campus Lost-and-Found Matcher

Match lost-item reports with found-item reports intelligently

Course	CET251 — Artificial Intelligence
Project Type	Course Project — Full Implementation
University	El Sewedy University of Technology
Backend	Insforge (insforge.dev)
Deployment	Railway
Academic Year	2024 / 2025

1. Executive Summary

The Campus Lost-and-Found Matcher is an AI-powered web application designed to solve a common yet persistent problem on university campuses: the manual, inefficient process of matching lost items with found ones. Students frequently misplace IDs, chargers, water bottles, notebooks, headphones, and keys — and the traditional bulletin board or email approach produces slow, unreliable results.

This system introduces an intelligent matching pipeline that accepts structured reports from both parties (the person who lost an item and the person who found one), then applies natural language processing and ranked similarity algorithms to suggest the most likely matches, each accompanied by a confidence score and a human-readable explanation of why the match was proposed.

The solution is built with a clean, production-grade UI/UX targeting real deployment within El Sewedy University, using Insforge as the backend platform and Railway for cloud deployment. The AI pipeline leverages TF-IDF vectorization, cosine similarity, and optional neural-network classification — all grounded in the core AI concepts of CET251.

2. Problem Statement

2.1 Background

University campuses see dozens of lost-and-found incidents daily. The current approach — posting on WhatsApp groups, pinning notes to boards, or filing forms with security offices — suffers from several critical issues:

- Reports are unstructured and written in inconsistent language.
- Manual comparison is time-consuming and error-prone.
- Items go unclaimed because the right person never sees the right post.
- There is no feedback mechanism or confidence signal for potential matches.

2.2 Scope of the Problem

Based on common campus survey data, the most frequently lost item categories include:

Item Category	Estimated Weekly Reports	Recovery Rate (Manual)
ID Cards	12–18	~35%
Chargers & Cables	8–14	~25%
Water Bottles	6–10	~20%
Notebooks	5–8	~30%
Headphones	4–7	~15%
Keys	5–9	~40%

2.3 Proposed Solution

An intelligent matching assistant that: (a) accepts structured lost and found reports through a polished web interface, (b) applies NLP-based similarity scoring to rank likely matches, and (c) presents results with confidence scores and reasoning to the student — all deployed as a live campus service.

3. Project Objectives

3.1 Primary Objectives

- Build a full-stack AI-powered lost-and-found web application accessible to all university students.
- Implement an intelligent matching engine using TF-IDF and cosine similarity to rank item matches.
- Support at least six item categories: ID card, charger, bottle, notebook, headphones, and keys.
- Generate match explanations that are human-readable and actionable.
- Achieve a dataset of 50–100 sample reports for testing and demonstration.

3.2 Secondary Objectives

- Integrate a text-based classifier (neural network add-on) to identify item type from free-text descriptions.
- Deploy the application live on Railway so real students can submit and browse reports.
- Design a UI/UX that is professional, mobile-friendly, and intuitive — appropriate for campus-wide rollout.
- Optionally support bilingual (Arabic/English) report submissions.
- Optionally implement duplicate report detection to keep the database clean.

3.3 Success Metrics

Metric	Minimum Target	Stretch Target
Sample reports in system	50	100+
Matching precision (top-3)	≥ 70%	≥ 85%
Item categories supported	6	8+
Languages supported	English	Arabic + English
Deployment	Railway (live)	Custom domain

4. System Architecture

4.1 High-Level Architecture

The system follows a three-tier architecture: a React-based frontend, an Insforge-hosted backend with REST APIs and database, and an AI/ML engine running as a Python service or serverless function.

Layer	Technology	Responsibility	Hosting
Frontend	React + Tailwind CSS	UI/UX, form submission, results display	Railway (static)
Backend API	Insforge	Authentication, CRUD, business logic	Insforge Cloud
AI Engine	Python (FastAPI)	Matching, scoring, classification	Railway (service)
Database	PostgreSQL / Insforge DB	Reports, users, match history	Insforge Cloud

4.2 Data Flow

The complete data flow for a matching request is as follows:

- Student submits a lost-item report via the frontend form (category, description, location, time, color).
- The frontend sends a POST request to the Insforge backend API.
- Insforge stores the report in the database and triggers the AI Engine endpoint.
- The AI Engine retrieves all found-item reports from the database and runs the matching pipeline.
- Ranked results (with confidence scores and explanations) are returned to the frontend.
- The student sees a clean ranked list of potential matches with actionable contact options.

4.3 Component Diagram (Textual)

```
[ Student Browser ]
        |  React SPA (form, results, history)
        ↓  HTTPS REST
[ Insforge Backend ]
        |  Auth • Reports API • DB (PostgreSQL)
        ↓  Internal API call
[ AI Matching Engine — Python/FastAPI on Railway ]
        |  TF-IDF • Cosine Similarity • Classifier • Ranker
        ↓  JSON ranked results
```

5. AI Design & Core Concepts

5.1 Agents & Environments

The matching system is modeled as a rational agent operating in a partially observable, static, and discrete environment:

- Percepts: structured lost/found report fields (description, category, location, time, color).
- Actions: return a ranked list of candidate matches with scores and explanations.
- Goal: maximize the probability that the true owner is within the top-3 results shown.
- Environment: the database of all current found-item reports, which changes as new reports arrive.

5.2 Matching Pipeline (Informed Search)

The core matching algorithm applies a multi-feature scoring function that combines several similarity signals:

Feature	Method	Weight	Notes
Text Description	TF-IDF + Cosine Similarity	0.40	Core NLP signal
Item Category	Exact match / penalty	0.25	Hard filter option
Color	Fuzzy string match	0.15	Normalized palette
Location	Token overlap	0.12	Building / floor keywords
Time Proximity	Temporal distance scoring	0.08	Decays over time

The composite score $S(\text{lost}, \text{found}) = \Sigma(\text{weight}_i \times \text{score}_i)$ is computed for every lost–found pair. Items are then sorted in descending order and the top-K results (default K=5) are returned.

5.3 Optional CSP Filtering

Before running the similarity pipeline, a Constraint Satisfaction Problem (CSP) filter can eliminate impossible matches:

- Category must match (hard constraint, configurable).
- Found date must be ≥ Lost date (temporal constraint).
- Location must be on the same campus zone (if zone metadata is provided).

CSP pre-filtering reduces the candidate pool and improves both speed and precision.

5.4 Neural-Network Add-On (Category Classifier)

A lightweight text classifier identifies item category from free-text descriptions, enabling users to skip the category dropdown if they prefer:

- Model: scikit-learn MLPClassifier or a simple Keras Sequential model.
- Input: TF-IDF vectors of the description text.
- Output: predicted category label (one of 6 classes) + confidence probability.
- Training data: 50–100 labeled descriptions, augmented synthetically.
- Fallback: if confidence < 0.6, the user is prompted to confirm the predicted category.

5.5 Explanation Generation

Each suggested match includes a plain-language explanation, constructed from the feature scores:

Trigger Condition	Example Explanation Text
Text sim > 0.7	"Descriptions are very similar — both mention a black laptop charger with a frayed cable."
Category match	"Both reports are categorized as Charger."
Color match	"Item colors match: both listed as blue."
Location match	"Both linked to the Engineering Building area."

6. Technical Stack

6.1 Frontend

- Framework: React 18 with Vite for fast builds.
- Styling: Tailwind CSS + custom design system for consistent, professional UI.
- State management: Zustand (lightweight, no boilerplate).
- HTTP client: Axios for Insforge API calls.
- Routing: React Router v6 for multi-page navigation.
- Icons: Lucide React for clean, consistent iconography.
- Animations: Framer Motion for smooth transitions and micro-interactions.

6.2 Backend (Insforge)

Insforge (insforge.dev) is used as the primary backend platform. It provides:

- Auto-generated REST APIs from a schema definition.
- Built-in authentication (student login via university email).
- Database hosting (PostgreSQL-compatible) with no additional configuration.

- Webhook support to trigger the AI Engine on new report submissions.

6.3 AI Engine

- Language: Python 3.11+
- Web framework: FastAPI (lightweight, async-ready, auto-documented).
- NLP: scikit-learn (TF-IDF, cosine similarity, MLPClassifier).
- Data manipulation: pandas for report preprocessing.
- Optional: TensorFlow Lite or ONNX for lightweight neural network inference.

6.4 Deployment (Railway)

- Frontend (React): deployed as a static site on Railway.
- AI Engine (FastAPI): deployed as a Railway service with auto-scaling.
- Environment variables: managed via Railway's secret store.
- CI/CD: GitHub Actions pipeline triggers Railway redeploys on push to main.
- Domain: custom subdomain (e.g., lostandfound.sewedy.edu.eg) planned for stretch goal.

6.5 Development Tools

Tool	Purpose	Version
VS Code	Primary IDE	Latest
Git / GitHub	Version control	—
Jupyter Notebook	AI pipeline prototyping	—
Postman	API testing	—
Figma	UI/UX wireframes & design system	—

7. Database Schema

7.1 Core Tables

Table: lost_reports

Column	Type	Constraints	Description
id	UUID	PK	Unique report identifier
student_id	UUID	FK → users	Reporter's user ID
category	ENUM	NOT NULL	ID card / charger / bottle / notebook / headphones / keys
description	TEXT	NOT NULL	Free-text description of the item
color	VARCHAR(50)	—	Primary color of the item
location_lost	VARCHAR(200)	NOT NULL	Where the item was last seen
time_lost	TIMESTAMP	NOT NULL	When the item was lost
status	ENUM	DEFAULT 'active'	active / matched / closed
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

Table: found_reports

Mirrors the structure of lost_reports with the following changes: location_found replaces location_lost, time_found replaces time_lost, and an additional field finder_contact (VARCHAR 200) stores how the finder can be reached.

Table: match_results

Column	Type	Constraints	Description
id	UUID	PK	Match record ID
lost_report_id	UUID	FK → lost_reports	The lost item report
found_report_id	UUID	FK → found_reports	The candidate found item
confidence_score	FLOAT	0.0–1.0	Overall match score
explanation	TEXT	—	Human-readable match reason
feature_scores	JSONB	—	Breakdown per feature (text, category, color, location, time)
created_at	TIMESTAMP	DEFAULT NOW()	When the match was computed

8. UI/UX Design

8.1 Design Philosophy

The UI targets a professional, consumer-grade experience — not a typical student project. The design principles are:

- Clarity: every screen has a single primary action; no cognitive overload.
- Speed: the report submission form must complete in under 60 seconds for a first-time user.
- Trust: confidence scores and match explanations are presented transparently, building user trust in the AI.
- Delight: subtle micro-animations and smooth transitions make the experience feel polished.
- Accessibility: WCAG 2.1 AA compliance — readable fonts, sufficient contrast, keyboard navigation.

8.2 Screen Inventory

Screen	Purpose	Key Components
Landing / Home	Introduce the service, quick stats (reports today, matches made), CTA buttons	Hero banner, stats cards, quick links
Report Lost Item	Multi-step form: category → description → details → review	Stepper, category picker, color swatch selector
Report Found Item	Same multi-step flow for found items	Same as above + contact info field
My Reports	Dashboard showing submitted reports and their match status	Status badges, match score indicators
Match Results	Ranked list of suggested matches for a given lost report	Match cards with score bars, explanation tooltips

Match Detail	Deep dive into a single match with full feature breakdown	Radar chart of feature scores, contact button
Admin Dashboard	Manage all reports, trigger re-matching, view analytics (stretch)	Data tables, charts, export

8.3 Design System

Color Palette

Role	Hex Value	Usage
Primary Blue	#1A56DB	CTAs, headings, active states, links
Accent Orange	#F97316	Confidence scores, highlights, notifications
Success Green	#10B981	Matched items, confirmation states
Warning Yellow	#F59E0B	Partial matches, pending status
Neutral Dark	#1E293B	Body text, card backgrounds
Neutral Mid	#475569	Secondary text, labels
Background	#F8FAFF	Page background, alternate table rows

Typography

Primary font: Inter (Google Fonts) — clean, highly legible at all sizes. Heading sizes: H1 = 32px, H2 = 24px, H3 = 18px. Body text: 16px / line-height 1.6. All font sizes scale responsively via Tailwind's responsive prefixes.

8.4 Responsive Design

The application is designed mobile-first. Breakpoints follow Tailwind's defaults: sm (640px), md (768px), lg (1024px), xl (1280px). On mobile, the report form collapses to a full-screen stepper. Match result cards stack vertically. Navigation collapses to a bottom tab bar.

9. Implementation Plan

9.1 Phased Roadmap

Phase	Duration	Deliverables	AI Concepts
1 — Foundation	Week 1–2	Insforge schema, DB setup, user auth, basic CRUD API for lost/found reports	Environment modeling
2 — AI Engine	Week 3–4	TF-IDF pipeline, cosine similarity ranker, feature scorer, explanation generator	Informed search, agents
3 — Frontend Core	Week 5–6	Report submission forms, My Reports page, basic match results page	—
4 — Integration	Week 7	Connect frontend to Insforge API + AI Engine, end-to-end testing with 50+ reports	CSP filtering

5 — UI Polish	Week 8	Design system implementation, animations, mobile responsiveness, accessibility	—
6 — Classifier Add-On	Week 9	Train category classifier, integrate into submission flow	Neural network, classification
7 — Deployment	Week 10	Railway deployment, environment config, smoke testing, demo data load	—
8 — Stretch Goals	Week 11–12	Arabic/English support, duplicate detection, image upload (optional)	—

9.2 Risk Analysis

Risk	Likelihood	Mitigation Strategy
Low matching accuracy with few reports	Medium	Synthetic data augmentation; tune weights using held-out test set
Insforge API rate limits	Low	Cache AI engine results; batch processing for bulk re-matching
Railway free-tier cold starts	Medium	Keep-alive pings every 5 min; upgrade if latency is critical
Arabic NLP complexity	High	Use cameltools or CAMeL NLP library; treat as optional stretch
Student adoption	Medium	Demo during class; integration with existing university WhatsApp groups

10. Testing Strategy

10.1 Unit Testing

- TF-IDF vectorizer: verify term frequencies for known input strings.
- Cosine similarity function: assert score = 1.0 for identical vectors, = 0.0 for orthogonal.
- Feature scorer: test each individual feature with known inputs and expected score ranges.
- Composite ranker: verify ranked output order matches expected priority.

10.2 Integration Testing

- Submit a lost report via the frontend and verify it appears in the Insforge database.
- Trigger the AI engine via the API and verify a ranked result is returned.
- Verify that explanations are generated and non-empty for every result.

10.3 End-to-End Matching Accuracy

A labeled test dataset of 30 lost–found pairs (10 true matches, 20 non-matches) will be used to evaluate:

- Precision @ 1: is the top result the true match?
- Precision @ 3: is the true match in the top 3 results?
- Mean Reciprocal Rank (MRR): average of 1/rank for true matches.

10.4 UI/UX Testing

- Manual testing across Chrome, Firefox, and Safari.
- Mobile testing on iOS Safari and Android Chrome.
- Lighthouse audit: target Performance ≥ 90, Accessibility ≥ 95.

11. Stretch Goals

11.1 Image Upload

Allow the finder to upload a photo of the found item. The system will extract dominant color from the image and use a pre-trained MobileNet or CLIP model to generate an image embedding. This embedding is added as an additional feature in the matching pipeline.

11.2 Arabic / English Bilingual Support

The submission form will accept both Arabic and English text. The AI Engine will detect language (using langdetect), apply language-appropriate preprocessing (CAMeL tools for Arabic, NLTK for English), and run TF-IDF in each language separately before combining similarity scores.

11.3 Duplicate Report Detection

Before saving a new report, the system will run a similarity check against existing open reports from the same user. If a report with similarity score > 0.85 already exists, the user is warned and prompted to update the existing report instead of creating a duplicate.

11.4 Match Notification System

When a new found-item report is submitted that matches an existing lost-item report above a threshold (score > 0.75), the original reporter receives an in-app notification and optionally an email via Insforge's notification hooks.

12. References & Resources

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Scikit-learn Documentation. TF-IDF and Cosine Similarity. https://scikit-learn.org
- Insforge Platform Documentation. https://insforge.dev
- Railway Deployment Docs. https://docs.railway.app
- Tailwind CSS Documentation. https://tailwindcss.com/docs
- React Documentation. https://react.dev
- FastAPI Documentation. https://fastapi.tiangolo.com
- CET251 Course Project Specification. El Sewedy University of Technology, 2024.